Pizza Sales Analysis

SQL project: 12 business questions answered with MySQL

Dataset: 21,350 orders | 48,620 line items | 32 pizza types | Jan-Dec 2015

21,350	$817,860	138	$38.31
Total orders	Total revenue	Avg pizzas per day	Avg order value

Project overview

This project analyses a year of pizza-restaurant sales using SQL. Four tables (**orders**, **orders_details**, **pizzas**, **pizza_types**) are joined to answer 12 questions on revenue, product mix, order timing and category performance. The SQL covers JOINs, GROUP BY, subqueries and window functions (`SUM() OVER`, `RANK() OVER`).

Key findings

- **Large pizzas lead:** L is the most ordered size (18,526 order lines, 38% of the total).
- **Two daily rush windows:** lunch (12-13h) and dinner (17-18h) bring in the most orders.
- **Balanced menu:** all four categories earn 24-27% of revenue. Classic leads with 26.9%.
- **Top 3 earners are all Chicken pizzas**: Thai, BBQ and California.
- **Stable sales:** monthly revenue stays in a $64K-$73K band, with no strong seasonality.

Database schema

Table	Columns	Rows
orders	order_id (PK), order_date, order_time	21,350
orders_details	order_details_id (PK), order_id, pizza_id, quantity	48,620
pizzas	pizza_id (PK), pizza_type_id, size, price	96
pizza_types	pizza_type_id (PK), name, category, ingredients	32

Relationships: orders.order_id = orders_details.order_id | orders_details.pizza_id = pizzas.pizza_id | pizzas.pizza_type_id = pizza_types.pizza_type_id

Tools: MySQL, SQL (joins, aggregations, subqueries, window functions). Charts generated from query results.

Part 1: Basic analysis

Q1. Total number of orders placed

```
SELECT COUNT(order_id) AS total_orders FROM orders;
```

Result: 21,350 orders.

Q2. Total revenue from pizza sales

```
SELECT ROUND(SUM(orders_details.quantity * pizzas.price), 2) AS total_sales
FROM orders_details
JOIN pizzas ON pizzas.pizza_id = orders_details.pizza_id;
```

Result: $817,860.05 in total revenue.

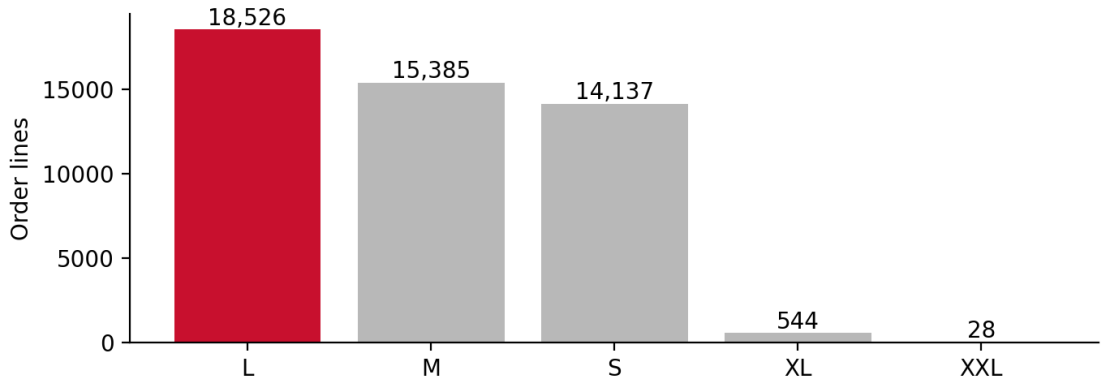
Q3. Highest-priced pizza

```
SELECT pizza_types.name, pizzas.price
FROM pizza_types
JOIN pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
ORDER BY pizzas.price DESC
LIMIT 1;
```

Result: The Greek Pizza at $35.95 (the XXL size).

Q4. Most common pizza size ordered

```
SELECT pizzas.size, COUNT(orders_details.order_details_id) AS order_count
FROM orders_details
JOIN pizzas ON orders_details.pizza_id = pizzas.pizza_id
GROUP BY pizzas.size
ORDER BY order_count DESC;
```



Insight: Large (L) is the clear favourite with 18,526 order lines. The XL and XXL sizes together make up barely 1% of orders, so their place on the menu is worth a review.

Part 2: Intermediate analysis

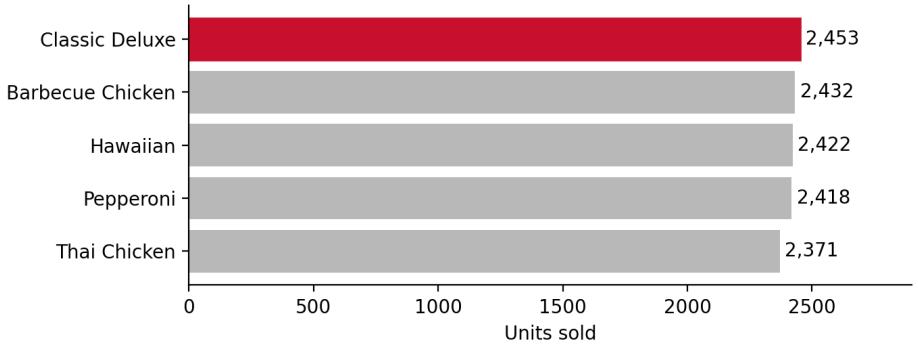
Q5. Total quantity ordered per pizza category

```
SELECT pizza_types.category, SUM(orders_details.quantity) AS total_quantity
FROM orders_details
JOIN pizzas ON orders_details.pizza_id = pizzas.pizza_id
JOIN pizza_types ON pizza_types.pizza_type_id = pizzas.pizza_type_id
GROUP BY category
ORDER BY total_quantity DESC;
```

Category	Pizzas ordered
Classic	14,888
Supreme	11,987
Veggie	11,649
Chicken	11,050

Q6. Top 5 most ordered pizza types

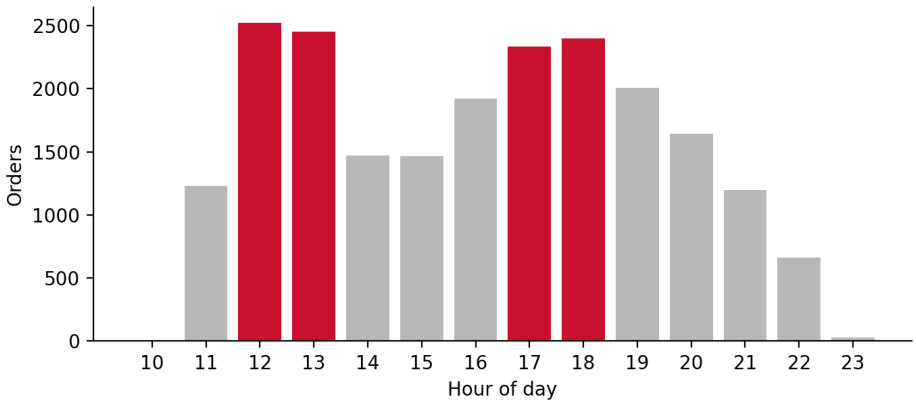
```
SELECT pizza_types.name, SUM(orders_details.quantity) AS total_quantity
FROM pizza_types
JOIN pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
JOIN orders_details ON orders_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY total_quantity DESC
LIMIT 5;
```



Insight: The top five are separated by only about 3% (2,453 vs 2,371 units), so demand is spread evenly across the menu rather than dominated by one hero product.

Q7. Distribution of orders by hour of the day

```
SELECT HOUR(order_time) AS hour, COUNT(order_id) AS order_count
FROM orders
GROUP BY HOUR(order_time)
ORDER BY hour;
```



Insight: Peaks at 12:00 (2,520 orders), 13:00 and 17:00-18:00 (highlighted). Staffing and prep should be scaled for these windows. The restaurant is effectively closed before 11:00 and after 23:00.

Part 3: Advanced analysis

Q8. Average number of pizzas ordered per day

```
SELECT ROUND(AVG(quantity), 0) AS avg_pizzas_ordered_per_day
FROM (SELECT orders.order_date, SUM(orders_details.quantity) AS quantity
      FROM orders
      JOIN orders_details ON orders.order_id = orders_details.order_id
      GROUP BY orders.order_date) AS order_quantity;
```

Result: about 138 pizzas per day (138.5 before rounding).

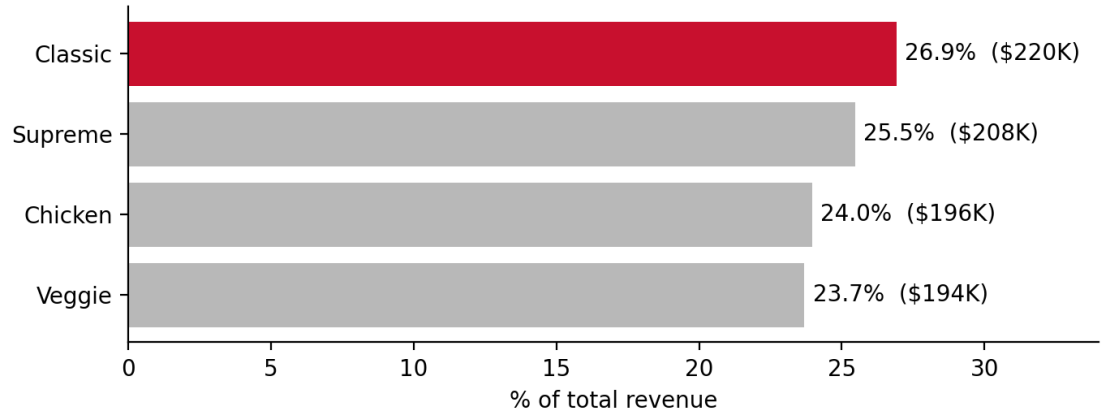
Q9. Top 3 pizza types by revenue

```
SELECT pizza_types.name, SUM(orders_details.quantity * pizzas.price) AS revenue
FROM pizza_types
JOIN pizzas ON pizzas.pizza_type_id = pizza_types.pizza_type_id
JOIN orders_details ON orders_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY revenue DESC
LIMIT 3;
```

Rank	Pizza	Revenue
1	The Thai Chicken Pizza	$43,434.25
2	The Barbecue Chicken Pizza	$42,768.00
3	The California Chicken Pizza	$41,409.50

Q10. Percentage contribution of each category to total revenue

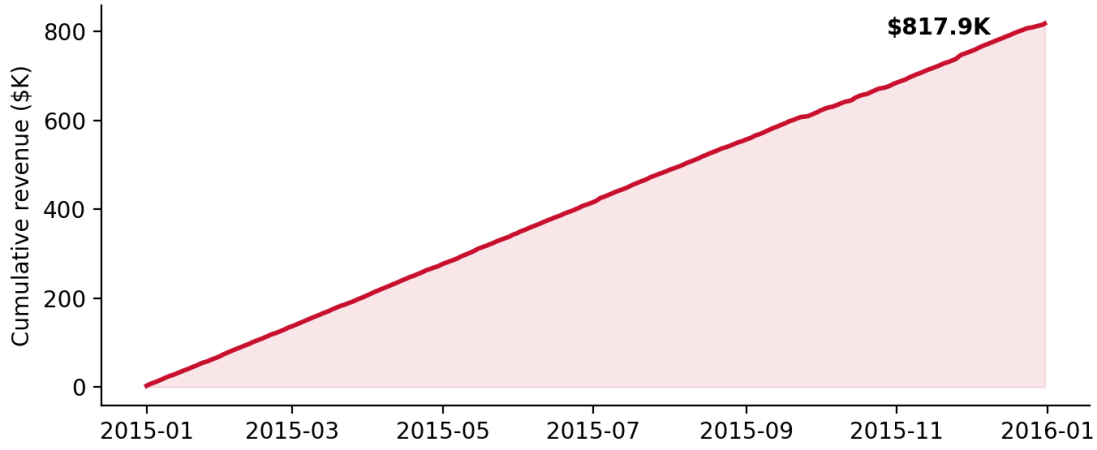
```
SELECT pizza_types.category,
  SUM(orders_details.quantity * pizzas.price) /
  (SELECT SUM(orders_details.quantity * pizzas.price)
   FROM orders_details JOIN pizzas ON pizzas.pizza_id = orders_details.pizza_id) * 100
  AS revenue_percentage
FROM pizza_types
JOIN pizzas ON pizzas.pizza_type_id = pizza_types.pizza_type_id
JOIN orders_details ON orders_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category
ORDER BY revenue_percentage DESC;
```



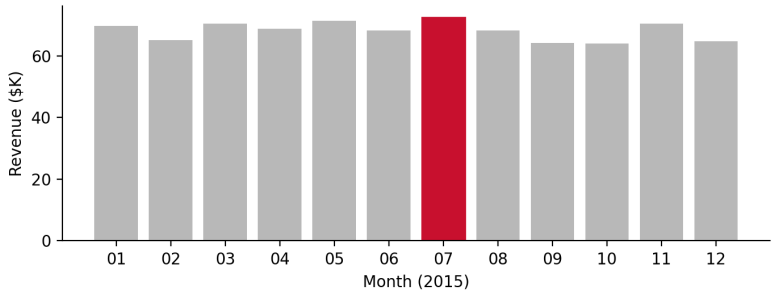
Insight: Revenue is split almost evenly across categories (24-27%). Classic earns the most, helped by the highest unit volume.

Q11. Cumulative revenue over time

```
SELECT order_date, SUM(revenue) OVER (ORDER BY order_date) AS cum_revenue
FROM (SELECT orders.order_date, SUM(orders_details.quantity * pizzas.price) AS revenue
      FROM orders_details
      JOIN pizzas ON orders_details.pizza_id = pizzas.pizza_id
      JOIN orders ON orders.order_id = orders_details.order_id
      GROUP BY orders.order_date) AS sales;
```



Insight: Revenue grows in an almost straight line, which shows steady demand all year. Month by month, July is the strongest at $72.6K and October the weakest at $64.0K.



Q12. Top 3 pizza types by revenue within each category

```
SELECT category, name, revenue FROM
 (SELECT category, name, revenue,
         RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rn
  FROM (SELECT pizza_types.category, pizza_types.name,
               SUM(orders_details.quantity * pizzas.price) AS revenue
        FROM pizza_types
        JOIN pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
        JOIN orders_details ON orders_details.pizza_id = pizzas.pizza_id
        GROUP BY pizza_types.category, pizza_types.name) AS a) AS b
WHERE rn <= 3;
```

Category	Rank	Pizza	Revenue
Chicken	1	The Thai Chicken Pizza	$43,434.25
Chicken	2	The Barbecue Chicken Pizza	$42,768.00
Chicken	3	The California Chicken Pizza	$41,409.50
Classic	1	The Classic Deluxe Pizza	$38,180.50
Classic	2	The Hawaiian Pizza	$32,273.25
Classic	3	The Pepperoni Pizza	$30,161.75
Supreme	1	The Spicy Italian Pizza	$34,831.25
Supreme	2	The Italian Supreme Pizza	$33,476.75
Supreme	3	The Sicilian Pizza	$30,940.50
Veggie	1	The Four Cheese Pizza	$32,265.70
Veggie	2	The Mexicana Pizza	$26,780.75
Veggie	3	The Five Cheese Pizza	$26,066.50

Conclusions & recommendations

• **Staff for the rushes.** Add capacity at 12-13h and 17-18h, when order volume is highest.
• **Promote the Chicken line.** It ranks last of the four categories by units but holds the top 3 revenue spots, thanks to higher prices. Bundles and upsells here can lift margin.
• **Review XL and XXL.** Together they are about 1% of order lines, so they add complexity and little revenue.
• **Smooth the quiet months.** September, October and December sit roughly $8K below the July peak. Targeted offers could close that gap.
• **Keep the broad menu.** Demand and revenue are evenly spread, so no single pizza or category is a risk.

Skills demonstrated

Relational schema design • multi-table JOINs • aggregate functions • subqueries • window functions (SUM OVER, RANK OVER) • date/time functions • translating query output into business insight.